Related Work Survey

How Agent-X Relates to Existing Systems

A survey of similar projects,
and where Agent-X fits in the landscape.

Generated from pathram2 composable sections
doc-28 (related-work) with 6 linked sections

Table of Contents

Part 1: QA by Graph Walking

Systems that answer questions by walking a graph of knowledge, rather than using neural networks.

Cyc (1984–present)

The longest-running knowledge base project. Over 25 million rules about the world, queryable by symbolic deduction. Like Agent-X, Cyc answers questions by reasoning over structured knowledge — no neural network at query time. Both systems treat knowledge as a typed graph of claims and relationships. Cyc's "microtheories" (context-specific rule sets) parallel Agent-X's domain nodes that scope which knowledge is relevant. Cyc's 1100 specialized inference modules that "raise their hand" for sub-problems mirror Agent-X's dispatch stage, which routes to derive/count/viveka/anumana based on detected signals.

Reference: Lenat, D.B. "CYC: A Large-Scale Investment in Knowledge Infrastructure." Communications of the ACM, 1995.

SNePS (Semantic Network Processing System)

A knowledge representation system where every node is a proposition and reasoning happens by network traversal. SNePS has three inference styles — formula-based, slot-based, and path-based — all of which find parallels in Agent-X: formula-based reasoning maps to tantra evaluation, slot-based maps to shabda key-value lookup, and path-based maps to graph walking across typed edges.

Reference: Shapiro, S.C. "SNePS: A Logic for Natural Language Understanding and Commonsense Reasoning." 2000.

OpenCog AtomSpace

A hypergraph where rules, queries, and knowledge all live in the same structure. This "self-describing" property is shared with Agent-X, where the pipeline nodes (construct, refine, dispatch, etc.) are themselves nodes in the same graph they operate on. OpenCog's pattern matcher walks the hypergraph to find matching substructures, similar to how Agent-X's PPR expand stage finds relevant knowledge subgraphs.

Reference: Goertzel, B. et al. "OpenCog: A Software Framework for Integrative Artificial General Intelligence." 2010.

Wikidata + SPARQL

The largest open knowledge graph, with over 100 billion triples queryable by graph pattern matching. Agent-X's question processing is analogous to SPARQL's Basic Graph Pattern matching — both find subgraph patterns in a knowledge store. The difference is scale and structure: Wikidata is broad and flat, Agent-X is deep and algebraically typed.

Reference: Vrandecic, D. and Krotzsch, M. "Wikidata: A Free Collaborative Knowledge Base." Communications of the ACM, 2014.

Part 2: Algebraic Knowledge Representation

Research on using algebra, category theory, and formal structures for knowledge representation.

Knowledgebra (Niu et al., 2022)

This paper proved that the semigroup is the most reasonable algebraic structure for general knowledge graph relations, by testing which algebraic properties (closure, associativity, identity, invertibility, commutativity) hold across real-world KGs. Agent-X's operation algebra satisfies all five properties — it is a group (has inverses via pratipaksha) and even a ring (has two interacting operations). This is possible because Agent-X's domain (math and physics formulas) has richer structure than general-purpose KGs: every operation genuinely has an inverse.

Reference: Niu, G. et al. "Knowledgebra: An Algebraic Learning Framework for Knowledge Graph." 2022. arXiv:2204.07328

RESCAL Tensor Factorization (Nickel et al.)

RESCAL models a knowledge graph as a 3D binary tensor $G \in \{0,1\}^{N \times N \times R}$ — exactly the same mathematical object as Agent-X's proof graph. Both systems represent knowledge as "which node connects to which node via which relation type." RESCAL then factorizes this tensor to find patterns; Agent-X walks it directly. The shared formulation validates that the 3D tensor is a natural representation for

typed knowledge graphs.

Reference: Nickel, M. et al. ”A Three-Way Model for Collective Learning on Multi-Relational Data.” ICML 2011.

DisCoCat (Coecke, Sadrzadeh, Clark)

Categorical compositional distributional semantics, implemented in the lambeq library. DisCoCat composes word meanings using tensor products and linear maps, where the composition follows grammatical structure. Agent-X shares the core insight: meaning is compositional, and the composition must respect grammatical structure. Both use non-commutative operations (order matters). DisCoCat works in continuous vector spaces for sentence similarity; Agent-X works in a discrete graph for question answering.

Reference: Coecke, B. et al. ”Mathematical Foundations for a Compositional Distributional Model of Meaning.” 2010. Also: Kartsaklis, D. et al. ”lambeq: An Efficient High-Level Python Library for Quantum NLP.” 2021. arXiv:2110.04236

Tarski’s Relation Algebra (1940s)

The mathematical foundation for treating binary relations algebraically: composition, converse (inverse), and Boolean operations. Agent-X’s visheshanam ring is a descendant of this tradition — the 10 edge types compose, have inverses (pratipaksha), and interact via distributivity. Tarski showed that relations can form an algebra; Agent-X applies this to a specific knowledge domain with 62 relation types.

Reference: Tarski, A. ”On the Calculus of Relations.” Journal of Symbolic Logic, 1941.

Knowledge Sheaves (Gebhart, 2023)

Uses sheaf theory (from algebraic topology) to structure knowledge graph embeddings. The key insight — that local consistency conditions on a knowledge graph can be expressed as a global algebraic object — resonates with Agent-X’s approach, where local edge properties (the 10 generators) compose to produce global graph behavior (the 52 dynamic dimensions).

Reference: Gebhart, T. ”Knowledge Sheaves: A Sheaf-Theoretic Framework for Knowledge Graph Embedding.” AISTATS 2023.

Groningen Knowledge Graphs (late 1980s)

One of the earliest projects to use typed edges specifically to enable algebraic operations on knowledge graphs. This is the direct intellectual ancestor of Agent-X’s core insight: that restricting edge types to a small typed set makes the graph amenable to algebraic reasoning.

Reference: van de Riet, R.P. et al. ”Knowledge Graphs.” University of Twente/Groningen, 1988.

Part 3: Formula Inversion

Systems that solve or manipulate mathematical formulas, and how their approaches relate to Agent-X.

Wolfram|Alpha

The premier computational knowledge engine. Wolfram solves and inverts formulas using classical symbolic algebra — pattern matching, rewrite rules, Grobner bases. Agent-X solves the same kinds of problems but by a completely different mechanism: reading the formula as a composition chain (krama) stored in the graph, and inverting by reversing the chain and looking up each operation’s inverse (pratipaksha) as a graph edge. Both systems agree that formula inversion is fundamental; they differ in mechanism (algebraic rewriting vs graph walking).

Reference: Wolfram, S. ”Wolfram|Alpha: Computational Knowledge Engine.” 2009.

MathGraph (Zhao et al., 2019)

A knowledge graph for automatically solving math exercises. Operations and constraints are graph nodes; solutions are found by topological traversal. Agent-X shares the insight that math knowledge can be a graph and computation can be graph traversal. MathGraph does forward computation (given inputs, compute output); Agent-X also does inverse computation (given output, find an input) by reversing the krama chain.

Reference: Zhao, W. et al. ”MathGraph: A Knowledge Graph for Automatically Solving Mathematical Exercises.” DASFAA 2019.

Biform Theory Graphs (Carette and Farmer, 2017)

Combines axiomatic theories (what is true) with algorithmic theories (how to compute) in a graph connected by meaning-preserving morphisms. Agent-X embodies this combination: the proof graph is axiomatic (nodes are claims, edges are typed relationships) and the tantras are algorithmic (compositions of operations that compute). The morphisms between them are the pipeline stages.

Reference: Carette, J. and Farmer, W.M. "Biform Theories in Chiron." 2017. arXiv:1704.02253

Part 4: Stratified Evaluation and Datalog

Systems using Datalog-style stratified fixpoint evaluation, and how they relate to Agent-X's pipeline.

Souffle

A high-performance Datalog engine that compiles logic programs to parallel C++. Agent-X's pipeline is structurally equivalent to Souffle's stratified evaluation: each pipeline stage (construct, assert, refine, expand, detect, dispatch, emit) is a stratum that only adds facts, never removes them (monotonicity). The refine stage's 13 sub-passes running to fixpoint directly parallel Souffle's semi-naive evaluation within a stratum.

Reference: Scholz, B. et al. "On Fast Large-Scale Program Analysis in Datalog." CC 2016.

Dyna (Jason Eisner, JHU)

A Datalog extension for NLP that replaces the Boolean semiring with arbitrary semirings (probabilities, Viterbi scores, etc.). Agent-X operates on the Boolean semiring ($\{0, 1\}$ with OR and AND), making it the simplest possible Dyna program. Dyna's insight that NLP can be expressed as weighted deduction over structured items validates Agent-X's approach of expressing NLP as Boolean deduction over a typed graph.

Reference: Eisner, J. and Filardo, N.W. "Dyna: Extending Datalog for Modern AI." Datalog 2.0, 2011.

LogicBlox

A commercial Datalog-based deductive database used in enterprise applications (supply chain, retail pricing). Shows that stratified Datalog evaluation scales to real-world problems. Agent-X applies the same evaluation strategy to a different domain (NLP over a proof graph).

Reference: Aref, M. et al. "Design and Implementation of the LogicBlox System." SIGMOD 2015.

Graal

A Java toolkit for ontological query answering using the chase algorithm (forward chaining rules to saturation). The chase is closely related to Agent-X's refine stage: both apply rules repeatedly until no new facts are derived. Graal uses existential rules (can assert new unknown entities); Agent-X's refine sub-passes enrich the question graph with typed triples.

Reference: Baget, J.F. et al. "Graal: A Toolkit for Query Answering with Existential Rules." 2015.

Part 5: Indian Logic in Computation

Formalizations and uses of Indian logic (Nyaya) in computational systems.

Navya-Nyaya Formal Language (Ghushe et al.)

The first systematic formalization of Navya-Nyaya technical language using Conceptual Graphs. This work showed that Indian logical categories can be rendered in graph-based notation. Agent-X takes the next step: rather than formalizing Indian logic for study, it uses Indian logical structures as the operational framework of a running system. The panchaavayava (five-limbed syllogism) is not described — it is the format every answer takes.

Reference: Ghushe, R. et al. "Later Nyaya Logic: Computational Aspects." Springer, 2014.

Matilal and Ingalls

The foundational Western scholarship on Navya-Nyaya. Matilal's "The Navya-Nyaya Doctrine of Negation" (1968) formalized abhava (absence) — the same concept that appears in Agent-X as rahita (the inverse of yukta). Ingalls' "Materials for the Study of Navya-Nyaya Logic" (1951) laid out the technical vocabulary that Agent-X's edge types echo: visheshanam (qualifier/relation), abheda (non-difference), pratipaksha (counter-thesis).

Reference: Matilal, B.K. ”The Navya-Nyaya Doctrine of Negation.” Harvard University Press, 1968. Ingalls, D.H.H. ”Materials for the Study of Navya-Nyaya Logic.” Harvard University Press, 1951.

Nyaya as Pre-Inferential Validation (PhilArchive)

A recent proposal to use Nyaya’s pramana (valid means of knowledge) as a validation layer for AI inference. Agent-X implements this: the detect stage validates which inference mode is appropriate (derive, count, viveka, anumana) before dispatching — this is pramana-driven selection, the system choosing its valid means of knowledge before reasoning.

Reference: Nyaya Pre-Inferential Validation Layer. PhilArchive, 2023.

Indian Logic and AI System Design (Matilal)

Proposes integrating Indian logic within frame structures for case-based reasoning. Agent-X achieves this integration differently: instead of frames, it uses a typed proof graph where the Indian logical categories are algebraic identities (shunya = additive identity, pratipaksha = group inverse, pratibodha = fixpoint).

Reference: ”Indian Logic and AI System Design.” Academia.edu.

Part 6: Novelty Analysis

What is unique about Agent-X — features where no existing system provides a match.

Graded Ring for Document Structure

No existing NLP system models input paragraphs as graded rings. The idea that each sentence is a grade, with additive accumulation within sentences and multiplicative selection across sentences, with ”respectively” as explicit distributivity — this has no precedent. The closest mathematical framework is graded rings in commutative algebra, but applied to polynomials, not language.

Formula Inversion by Graph Walking

Wolfram|Alpha inverts formulas using symbolic algebra (rewrite rules). Agent-X inverts by reading the formula as a composition chain from the graph, reversing it, and replacing each operation with its graph-declared inverse. No other system combines: (1) formulas as s-expression compositions stored as graph edges, (2) inverse operations declared as graph edges on the same nodes, (3) inversion as a pure graph walk requiring no algebraic engine.

Philosophical Ontology as Algebraic Identity

No system maps philosophical concepts to algebraic identities where the concepts are load-bearing:

Concept	Algebraic role	What breaks without it
shunya (emptiness)	additive identity	Count fold has no starting value
pratipaksha (opposition)	group inverse	Formulas cannot be inverted
pratibodha (awakening)	fixpoint iteration	Refine stage never converges
swarupa (self-nature)	multiplicative identity	Identity morphism is missing
viraam (cessation)	grade boundary	Sentence boundaries vanish

These are not decorative labels. Removing shunya from the graph breaks the fold. Removing pratipaksha breaks inversion. The philosophy IS the algebra.

The Combination

Each individual feature has partial precedents (Cyc for graph QA, RESCAL for 3D tensors, Souffle for stratified evaluation, Matilal for Indian logic formalization). No system combines all of them into a single architecture where the proof graph, the algebraic edge ring, the NLP pipeline, the formula system, and the philosophical ontology are the same structure.

Key references for the full survey: - Cyc: Lenat 1995 - SNePS: Shapiro 2000 - OpenCog: Goertzel 2010 - RESCAL: Nickel et al. 2011 - Knowledgebra: Niu et al. 2022 - DisCoCat: Coecke et al. 2010 - Tarski: 1941 - Dyna: Eisner and Filardo 2011 - Souffle: Scholz et al. 2016 - MathGraph: Zhao et al. 2019 - Navya-Nyaya: Ghushe et al. 2014, Matilal 1968, Ingalls 1951 - Knowledge Sheaves: Gebhart 2023 - Groningen KGs: van de Riet 1988